

Pontificia Universidade Catolica de Goias

Escola Politecnica e de Artes
Curso de Ciencia da Computacao

Atividade Final

Paradigmas de Linguagens de Programacao

Sistema de Analise de Desempenho Academico

Alunos:

Amanda Barbosa Ferreira Agapito
Guilherme Jose da Silva
Matheus Silva Pains

Professor: Roussian de Ramos Alves Gaioso

Goiania, Goias
12 de Junho de 2026

1. Descricao do Problema

O sistema recebe dados de alunos contendo nome, matricula, tres notas e frequencia. A media e calculada como: $media = (nota1 + nota2 + nota3) / 3$. O aluno e aprovado se $media \geq 6.0$ E $frequencia \geq 75\%$. Caso contrario, e reprovado.

Dados de Entrada (alunos.csv)

```
Ana;2024001;8.0;7.5;9.0;85
Bruno;2024002;5.0;6.0;4.5;80
Carlos;2024003;7.0;6.5;8.0;60
Daniela;2024004;9.0;9.5;10.0;95
Eduardo;2024005;3.0;4.0;2.5;70
Fernanda;2024006;6.0;6.0;6.0;75
```

Resultado Esperado

Ana: media 8.17, freq 85 -> Aprovado | Bruno: media 5.17, freq 80 -> Reprovado

Carlos: media 7.17, freq 60 -> Reprovado | Daniela: media 9.50, freq 95 -> Aprovado

Eduardo: media 3.17, freq 70 -> Reprovado | Fernanda: media 6.00, freq 75 -> Aprovado

2. Linguagens Escolhidas

- C (imperativo) -- linguagem procedural por excelencia, com controle explicito de memoria, structs, ponteiros e funcoes. Ideal para o paradigma imperativo puro.

- Python (orientado a objetos) -- classes com encapsulamento (name mangling), metodos de negocio e organizacao em objetos.

- Python (funcional) -- map, filter, reduce, lambdas e dicionarios imutaveis para um estilo predominantemente funcional.

- Prolog (logico) -- linguagem declarativa baseada em fatos e regras.

C foi escolhida para a versao imperativa por ser a linguagem sugerida pelo enunciado e a mais representativa do paradigma procedural. Python foi utilizada nas versoes OO e funcional por sua versatilidade. Prolog foi mantido por ser a referencia natural do paradigma logico.

3. Parte 1 -- Implementacao Imperativa (C)

Codigo-fonte: parte1_imperativo.c

```

1 | /* =====
2 |  * Parte 1 -- Implementacao Imperativa
3 |  * Sistema de Analise de Desempenho Academico
4 |  * Paradigma: Imperativo/Procedural (C)
5 |  * ===== */
6 |
7 | #include <stdio.h>
8 | #include <stdlib.h>
9 | #include <string.h>
10 |
11 | #define MAX_ALUNOS 50
12 | #define MAX_NOME 50
13 |
14 | /* ----- Estrutura de dados ----- */
15 |
16 | typedef struct {
17 |     char nome[MAX_NOME];
18 |     char matricula[20];
19 |     float nota1;
20 |     float nota2;
21 |     float nota3;
22 |     int frequencia;
23 | } Aluno;
24 |
25 | /* ----- Funcoes procedurais ----- */
26 |
27 | float calcular_media(float n1, float n2, float n3) {
28 |     float media;
29 |     media = (n1 + n2 + n3) / 3.0;
30 |     return media;
31 | }
32 |
33 | int verificar_aprovacao(float media, int frequencia) {
34 |     if (media >= 6.0 && frequencia >= 75) {
35 |         return 1; /* aprovado */
36 |     } else {
37 |         return 0; /* reprovado */
38 |     }
39 | }
40 |
41 | int ler_arquivo(const char *caminho, Aluno alunos[]) {
42 |     FILE *arquivo;
43 |     char linha[256];
44 |     int total = 0;
45 |
46 |     arquivo = fopen(caminho, "r");
47 |     if (arquivo == NULL) {
48 |         printf("Erro ao abrir o arquivo: %s\n", caminho);
49 |         return 0;
50 |     }
51 |
52 |     while (fgets(linha, sizeof(linha), arquivo) != NULL) {
53 |         if (strlen(linha) <= 1) {
54 |             continue;
55 |         }
56 |         /* Remove o \n final */
57 |         linha[strcspn(linha, "\r\n")] = '\0';
58 |
59 |         /* Parsing manual com sscanf e strtok */
60 |         char *token;
61 |         token = strtok(linha, ";");
62 |         if (token != NULL) strcpy(alunos[total].nome, token);
63 |
64 |         token = strtok(NULL, ";");

```

```

65 |         if (token != NULL) strcpy(alunos[total].matricula, token);
66 |
67 |         token = strtok(NULL, ";");
68 |         if (token != NULL) alunos[total].nota1 = atof(token);
69 |
70 |         token = strtok(NULL, ";");
71 |         if (token != NULL) alunos[total].nota2 = atof(token);
72 |
73 |         token = strtok(NULL, ";");
74 |         if (token != NULL) alunos[total].nota3 = atof(token);
75 |
76 |         token = strtok(NULL, ";");
77 |         if (token != NULL) alunos[total].frequencia = atoi(token);
78 |
79 |         total = total + 1;
80 |
81 |         if (total >= MAX_ALUNOS) {
82 |             break;
83 |         }
84 |     }
85 |
86 |     fclose(arquivo);
87 |     return total;
88 | }
89 |
90 | void gerar_relatorio(Aluno alunos[], int total) {
91 |     int i;
92 |     int total_aprovados = 0;
93 |     int total_reprovados = 0;
94 |     float media;
95 |     int aprovado;
96 |
97 |     printf("=====\n");
98 |     printf("RELATORIO DE DESEMPENHO ACADEMICO - VERSAO IMPERATIVA (C)\n");
99 |     printf("=====\n");
100 |     printf("%-12s %-12s %-6s %-6s %-6s %-8s %-6s %s\n",
101 |           "Nome", "Matricula", "N1", "N2", "N3", "Media", "Freq", "Situacao");
102 |     printf("-----\n");
103 |
104 |     i = 0;
105 |     while (i < total) {
106 |         media = calcular_media(alunos[i].nota1, alunos[i].nota2, alunos[i].nota3);
107 |         aprovado = verificar_aprovacao(media, alunos[i].frequencia);
108 |
109 |         if (aprovado == 1) {
110 |             total_aprovados = total_aprovados + 1;
111 |         } else {
112 |             total_reprovados = total_reprovados + 1;
113 |         }
114 |
115 |         printf("%-12s %-12s %-6.1f %-6.1f %-6.1f %-8.2f %-6d %s\n",
116 |               alunos[i].nome,
117 |               alunos[i].matricula,
118 |               alunos[i].nota1,
119 |               alunos[i].nota2,
120 |               alunos[i].nota3,
121 |               media,
122 |               alunos[i].frequencia,
123 |               aprovado ? "Aprovado" : "Reprovado");
124 |
125 |         i = i + 1;
126 |     }
127 |
128 |     printf("-----\n");
129 |     printf("Total de alunos: %d\n", total);
130 |     printf("Aprovados: %d\n", total_aprovados);
131 |     printf("Reprovados: %d\n", total_reprovados);
132 |     printf("=====\n");
133 | }
134 |
135 | /* ----- Programa principal ----- */
136 |
137 | int main(void) {
138 |     Aluno alunos[MAX_ALUNOS];

```

```
139 |     int total;
140 |
141 |     total = ler_arquivo("alunos.csv", alunos);
142 |
143 |     if (total > 0) {
144 |         gerar_relatorio(alunos, total);
145 |     }
146 |
147 |     return 0;
148 | }
```

Saida da Execucao

```
=====
RELATORIO DE DESEMPENHO ACADEMICO - VERSAO IMPERATIVA (C)
=====
Nome           Matricula    N1     N2     N3     Media    Freq    Situacao
-----
Ana            2024001      8.0    7.5    9.0    8.17     85      Aprovado
Bruno          2024002      5.0    6.0    4.5    5.17     80      Reprovado
Carlos         2024003      7.0    6.5    8.0    7.17     60      Reprovado
Daniela        2024004      9.0    9.5    10.0   9.50     95      Aprovado
Eduardo        2024005      3.0    4.0    2.5    3.17     70      Reprovado
Fernanda       2024006      6.0    6.0    6.0    6.00     75      Aprovado
-----
Total de alunos: 6
Aprovados: 3
Reprovados: 3
=====
```

4. Parte 2 -- Implementacao Orientada a Objetos (Python)

Codigo-fonte: parte2_oo.py

```

1 | # =====
2 | # Parte 2 -- Implementacao Orientada a Objetos
3 | # Sistema de Analise de Desempenho Academico
4 | # Paradigma: Orientado a Objetos (Python)
5 | # =====
6 |
7 | import os
8 |
9 |
10 | class Aluno:
11 |     """Representa um aluno com seus dados academicos."""
12 |
13 |     def __init__(self, nome, matricula, nota1, nota2, nota3, frequencia):
14 |         self.__nome = nome
15 |         self.__matricula = matricula
16 |         self.__nota1 = nota1
17 |         self.__nota2 = nota2
18 |         self.__nota3 = nota3
19 |         self.__frequencia = frequencia
20 |
21 |     # ----- Getters (encapsulamento) -----
22 |
23 |     def get_nome(self):
24 |         return self.__nome
25 |
26 |     def get_matricula(self):
27 |         return self.__matricula
28 |
29 |     def get_nota1(self):
30 |         return self.__nota1
31 |
32 |     def get_nota2(self):
33 |         return self.__nota2
34 |
35 |     def get_nota3(self):
36 |         return self.__nota3
37 |
38 |     def get_frequencia(self):
39 |         return self.__frequencia
40 |
41 |     # ----- Metodos de negocio -----
42 |
43 |     def calcular_media(self):
44 |         """Calcula e retorna a media aritmetica das tres notas."""
45 |         return (self.__nota1 + self.__nota2 + self.__nota3) / 3
46 |
47 |     def verificar_aprovacao(self):
48 |         """Retorna 'Aprovado' se media >= 6.0 e frequencia >= 75, senao 'Reprovado'."""
49 |         media = self.calcular_media()
50 |         if media >= 6.0 and self.__frequencia >= 75:
51 |             return "Aprovado"
52 |         return "Reprovado"
53 |
54 |     def __str__(self):
55 |         media = self.calcular_media()
56 |         situacao = self.verificar_aprovacao()
57 |         return (f"{self.__nome:<12} {self.__matricula:<12} "
58 |                 f"{self.__nota1:<6.1f} {self.__nota2:<6.1f} {self.__nota3:<6.1f} "
59 |                 f"{media:<8.2f} {self.__frequencia:<6} {situacao}")
60 |
61 |
62 | class Turma:
63 |     """Gerencia uma colecao de alunos e gera relatorios."""
64 |

```

```

65 |     def __init__(self):
66 |         self.__alunos = []
67 |
68 |     def carregar_de_arquivo(self, caminho):
69 |         """Le o arquivo CSV e popula a lista de alunos."""
70 |         with open(caminho, "r", encoding="utf-8") as arquivo:
71 |             for linha in arquivo:
72 |                 linha = linha.strip()
73 |                 if linha:
74 |                     partes = linha.split(";")
75 |                     aluno = Aluno(
76 |                         nome=partes[0],
77 |                         matricula=partes[1],
78 |                         nota1=float(partes[2]),
79 |                         nota2=float(partes[3]),
80 |                         nota3=float(partes[4]),
81 |                         frequencia=int(partes[5])
82 |                     )
83 |                     self.__alunos.append(aluno)
84 |
85 |     def obter_aprovados(self):
86 |         """Retorna a lista de alunos aprovados."""
87 |         aprovados = []
88 |         for aluno in self.__alunos:
89 |             if aluno.verificar_aprovacao() == "Aprovado":
90 |                 aprovados.append(aluno)
91 |         return aprovados
92 |
93 |     def obter_reprovados(self):
94 |         """Retorna a lista de alunos reprovados."""
95 |         reprovados = []
96 |         for aluno in self.__alunos:
97 |             if aluno.verificar_aprovacao() == "Reprovado":
98 |                 reprovados.append(aluno)
99 |         return reprovados
100 |
101 |     def gerar_relatorio(self):
102 |         """Imprime o relatorio completo de desempenho."""
103 |         print("=" * 70)
104 |         print("RELATORIO DE DESEMPENHO ACADÊMICO -- VERSAO ORIENTADA A OBJETOS")
105 |         print("=" * 70)
106 |         print(f"{'Nome':<12} {'Matricula':<12} {'N1':<6} {'N2':<6} {'N3':<6} {'Media':<8} {'Freq':<6} ...")
107 |         print("-" * 70)
108 |
109 |         for aluno in self.__alunos:
110 |             print(aluno)
111 |
112 |         aprovados = self.obter_aprovados()
113 |         reprovados = self.obter_reprovados()
114 |
115 |         print("-" * 70)
116 |         print(f"Total de alunos: {len(self.__alunos)}")
117 |         print(f"Aprovados: {len(aprovados)}")
118 |         print(f"Reprovados: {len(reprovados)}")
119 |         print("=" * 70)
120 |
121 |
122 | # ----- Programa principal -----
123 |
124 | caminho = os.path.join(os.path.dirname(os.path.abspath(__file__)), "alunos.csv")
125 | turma = Turma()
126 | turma.carregar_de_arquivo(caminho)
127 | turma.gerar_relatorio()

```

Saida da Execucao

```

=====
RELATORIO DE DESEMPENHO ACADEMICO -- VERSAO ORIENTADA A OBJETOS
=====
Nome           Matricula   N1      N2      N3      Media   Freq   Situacao
-----

```

Atividade Final - Paradigmas de Linguagens de Programacao

Ana	2024001	8.0	7.5	9.0	8.17	85	Aprovado
Bruno	2024002	5.0	6.0	4.5	5.17	80	Reprovado
Carlos	2024003	7.0	6.5	8.0	7.17	60	Reprovado
Daniela	2024004	9.0	9.5	10.0	9.50	95	Aprovado
Eduardo	2024005	3.0	4.0	2.5	3.17	70	Reprovado
Fernanda	2024006	6.0	6.0	6.0	6.00	75	Aprovado

Total de alunos: 6

Aprovados: 3

Reprovados: 3

=====

5. Parte 3 -- Implementacao Funcional (Python)

Codigo-fonte: parte3_funcional.py

```

1 | # =====
2 | # Parte 3 -- Implementacao Funcional
3 | # Sistema de Analise de Desempenho Academico
4 | # Paradigma: Funcional (Python)
5 | # =====
6 |
7 | import os
8 | from functools import reduce
9 |
10 |
11 | # ----- Funções puras -----
12 |
13 | def parse_linha(linha):
14 |     """Converte uma linha CSV em um dicionario imutavel de dados do aluno."""
15 |     partes = linha.strip().split(";")
16 |     return {
17 |         "nome": partes[0],
18 |         "matricula": partes[1],
19 |         "nota1": float(partes[2]),
20 |         "nota2": float(partes[3]),
21 |         "nota3": float(partes[4]),
22 |         "frequencia": int(partes[5])
23 |     }
24 |
25 |
26 | def calcular_media(aluno):
27 |     """Funcao pura: retorna a media aritmetica das tres notas."""
28 |     return (aluno["nota1"] + aluno["nota2"] + aluno["nota3"]) / 3
29 |
30 |
31 | def verificar_aprovacao(aluno):
32 |     """Funcao pura: retorna True se o aluno esta aprovado."""
33 |     return calcular_media(aluno) >= 6.0 and aluno["frequencia"] >= 75
34 |
35 |
36 | def gerar_resultado(aluno):
37 |     """Funcao pura: retorna um novo dicionario com media e situacao adicionados."""
38 |     media = calcular_media(aluno)
39 |     situacao = "Aprovado" if verificar_aprovacao(aluno) else "Reprovado"
40 |     return {**aluno, "media": media, "situacao": situacao}
41 |
42 |
43 | def contar_por_situacao(accumulator, resultado):
44 |     """Funcao para reduce: conta aprovados e reprovados."""
45 |     if resultado["situacao"] == "Aprovado":
46 |         return (accumulator[0] + 1, accumulator[1])
47 |     else:
48 |         return (accumulator[0], accumulator[1] + 1)
49 |
50 |
51 | # ----- Entrada -----
52 |
53 | def ler_dados(caminho):
54 |     """Le o arquivo e retorna lista de dicionarios (separacao entrada/processamento)."""
55 |     with open(caminho, "r", encoding="utf-8") as arquivo:
56 |         linhas = arquivo.readlines()
57 |     return list(map(parse_linha, filter(lambda l: l.strip() != "", linhas)))
58 |
59 |
60 | # ----- Saida -----
61 |
62 | def imprimir_relatorio(resultados):
63 |     """Imprime o relatorio formatado (separacao de saida)."""
64 |     print("=" * 70)

```

```

65 |     print("RELATORIO DE DESEMPENHO ACADÊMICO -- VERSAO FUNCIONAL")
66 |     print("=" * 70)
67 |     print(f"{'Nome':<12} {'Matricula':<12} {'N1':<6} {'N2':<6} {'N3':<6} {'Media':<8} {'Freq':<6} {'Si...
68 |     print("-" * 70)
69 |
70 |     for r in resultados:
71 |         print(f"{r['nome']:<12} {r['matricula']:<12} "
72 |               f"{r['nota1']:<6.1f} {r['nota2']:<6.1f} {r['nota3']:<6.1f} "
73 |               f"{r['media']:<8.2f} {r['frequencia']:<6} {r['situacao']}")
74 |
75 |     aprovados, reprovados = reduce(contar_por_situacao, resultados, (0, 0))
76 |
77 |     print("-" * 70)
78 |     print(f"Total de alunos: {len(resultados)}")
79 |     print(f"Aprovados: {aprovados}")
80 |     print(f"Reprovados: {reprovados}")
81 |     print("=" * 70)
82 |
83 |
84 | # ----- Pipeline funcional -----
85 |
86 | caminho = os.path.join(os.path.dirname(os.path.abspath(__file__)), "alunos.csv")
87 |
88 | # Entrada
89 | alunos = ler_dados(caminho)
90 |
91 | # Processamento (map aplica gerar_resultado a cada aluno)
92 | resultados = list(map(gerar_resultado, alunos))
93 |
94 | # Exemplo de filter: listar apenas aprovados
95 | aprovados = list(filter(lambda r: r["situacao"] == "Aprovado", resultados))
96 |
97 | # Saida
98 | imprimir_relatorio(resultados)
99 |
100 | print("\n--- Alunos Aprovados (via filter) ---")
101 | for a in aprovados:
102 |     print(f"  {a['nome']} -- Media: {a['media']:.2f}")

```

Saida da Execuciao

```

=====
RELATORIO DE DESEMPENHO ACADÊMICO -- VERSAO FUNCIONAL
=====
Nome           Matricula    N1     N2     N3     Media    Freq    Situacao
-----
Ana            2024001      8.0    7.5    9.0     8.17     85     Aprovado
Bruno          2024002      5.0    6.0    4.5     5.17     80     Reprovado
Carlos         2024003      7.0    6.5    8.0     7.17     60     Reprovado
Daniela        2024004      9.0    9.5    10.0    9.50     95     Aprovado
Eduardo        2024005      3.0    4.0    2.5     3.17     70     Reprovado
Fernanda       2024006      6.0    6.0    6.0     6.00     75     Aprovado
-----
Total de alunos: 6
Aprovados: 3
Reprovados: 3
=====

--- Alunos Aprovados (via filter) ---
Ana -- Media: 8.17
Daniela -- Media: 9.50
Fernanda -- Media: 6.00

```

6. Parte 4 -- Implementacao Logica (Prolog)

Codigo-fonte: parte4_logico.pl

```

1 | % =====
2 | % Parte 4 -- Implementacao Logica
3 | % Sistema de Analise de Desempenho Academico
4 | % Paradigma: Logico/Declarativo (Prolog)
5 | % =====
6 |
7 | % ----- Fatos: aluno(Nome, Matricula, Nota1, Nota2, Nota3, Frequencia) -----
8 |
9 | aluno(ana,      2024001, 8.0, 7.5, 9.0, 85).
10 | aluno(bruno,    2024002, 5.0, 6.0, 4.5, 80).
11 | aluno(carlos,   2024003, 7.0, 6.5, 8.0, 60).
12 | aluno(daniela,  2024004, 9.0, 9.5, 10.0, 95).
13 | aluno(eduardo,  2024005, 3.0, 4.0, 2.5, 70).
14 | aluno(fernanda, 2024006, 6.0, 6.0, 6.0, 75).
15 |
16 | % ----- Regra: calcular media -----
17 |
18 | media(Nome, Media) :-
19 |     aluno(Nome, _, N1, N2, N3, _),
20 |     Media is (N1 + N2 + N3) / 3.
21 |
22 | % ----- Regra: verificar frequencia suficiente -----
23 |
24 | frequencia_suficiente(Nome) :-
25 |     aluno(Nome, _, _, _, _, Freq),
26 |     Freq >= 75.
27 |
28 | % ----- Regra: determinar aprovacao -----
29 |
30 | aprovado(Nome) :-
31 |     media(Nome, Media),
32 |     Media >= 6.0,
33 |     frequencia_suficiente(Nome).
34 |
35 | % ----- Regra: determinar reprovacao -----
36 |
37 | reprovado(Nome) :-
38 |     aluno(Nome, _, _, _, _, _),
39 |     \+ aprovado(Nome).
40 |
41 | % ----- Regra: exibir situacao de um aluno -----
42 |
43 | situacao(Nome, Matricula, Media, Freq, Situacao) :-
44 |     aluno(Nome, Matricula, _, _, _, Freq),
45 |     media(Nome, Media),
46 |     (aprovado(Nome) -> Situacao = aprovado ; Situacao = reprovado).
47 |
48 | % ----- Consultas uteis -----
49 | % Para consultar no interpretador Prolog:
50 | %
51 | % Listar todos os aprovados:
52 | % ?- aprovado(X).
53 | %
54 | % Listar todos os reprovados:
55 | % ?- reprovado(X).
56 | %
57 | % Ver situacao de um aluno:
58 | % ?- situacao(ana, Mat, Media, Freq, Sit).
59 | %
60 | % Ver situacao de todos:
61 | % ?- situacao(Nome, Mat, Media, Freq, Sit).
62 |
63 | % ----- Regra para imprimir relatorio -----
64 |

```

```

65 | imprimir_aluno(Nome) :-
66 |     situacao(Nome, Matricula, Media, Freq, Situacao),
67 |     format("~w | ~w | Media: ~2f | Freq: ~w | ~w~n",
68 |           [Nome, Matricula, Media, Freq, Situacao])).
69 |
70 | relatorio :-
71 |     write('====='), nl,
72 |     write('RELATORIO - VERSAO LOGICA (PROLOG)'), nl,
73 |     write('====='), nl,
74 |     forall(aluno(Nome, _, _, _, _), imprimir_aluno(Nome)),
75 |     write('====='), nl.

```

Exemplo de Consultas e Saida

```

?- aprovado(X).
X = ana ;
X = daniela ;
X = fernanda.

?- reprovado(X).
X = bruno ;
X = carlos ;
X = eduardo.

?- situacao(ana, Mat, Media, Freq, Sit).
Mat = 2024001, Media = 8.166666666666666, Freq = 85, Sit = aprovado.

?- relatorio.
=====
RELATORIO - VERSAO LOGICA (PROLOG)
=====
ana | 2024001 | Media: 8.17 | Freq: 85 | aprovado
bruno | 2024002 | Media: 5.17 | Freq: 80 | reprovado
carlos | 2024003 | Media: 7.17 | Freq: 60 | reprovado
daniela | 2024004 | Media: 9.50 | Freq: 95 | aprovado
eduardo | 2024005 | Media: 3.17 | Freq: 70 | reprovado
fernanda | 2024006 | Media: 6.00 | Freq: 75 | aprovado
=====
true.

```

7. Comparacao de Legibilidade

Clareza dos nomes:

Todas as versoes Python usam nomes descritivos em portugues. Prolog usa atomos curtos (ana, aprovado), concisos mas claros.

Organizacao:

A OO e a mais organizada: dados e logica encapsulados em classes. A funcional separa entrada/processamento/saida. A imperativa e linear e direta.

Detalhes sintaticos:

Prolog tem a menor quantidade de sintaxe. Python imperativo exige mais linhas por usar while e indices explicitos.

Fluxo do programa:

A imperativa e a mais facil de seguir passo a passo. A funcional exige entender composicao de funcoes. Prolog nao tem fluxo explicito.

Localizacao das regras:

Em Prolog, cada regra e uma clausula isolada. Na OO, estao nos metodos. Na funcional, em funcoes nomeadas.

Conclusao: *Para iniciantes, a imperativa e mais legivel. Para projetos maiores, a OO oferece melhor organizacao. Prolog e o mais conciso, mas exige familiaridade.*

8. Comparacao de Facilidade de Escrita

Quantidade de codigo:

Prolog: ~40 linhas. Funcional: ~70 linhas. Imperativa: ~80 linhas. OO: ~100 linhas.

Dificuldade sintatica:

Python e uniforme nos tres paradigmas. Prolog exige aprender unificacao e backtracking.

Manipulacao de listas:

A funcional e a mais natural com map/filter. A imperativa usa indices manuais. A OO itera com for.

Estruturas auxiliares:

A OO exige criar classes. A funcional usa dicionarios. A imperativa usa listas de listas. Prolog usa fatos diretamente.

Conclusao: *A funcional Python foi a mais rapida de implementar. Prolog e conciso, mas exige mudanca de mentalidade.*

9. Comparacao de Confiabilidade

Verificacao de tipos:

Python tem tipagem dinamica -- erros de tipo so aparecem em tempo de execucao. Prolog nao tem tipos explicitos.

Erros em tempo de execucao:

A imperativa e mais suscetivel a erros de indice (alunos[i][5]). A OO protege dados com encapsulamento. A funcional

reduz efeitos colaterais com funcoes puras.

Acesso a memoria:

Python e Prolog nao permitem acesso direto a memoria, eliminando riscos de buffer overflow (diferente de C).

Testabilidade:

A funcional e a mais testavel: funcoes puras podem ser testadas isoladamente. A OO permite testes por metodo.

Conclusao: A funcional oferece maior confiabilidade por funcoes puras. A OO vem em segundo pelo encapsulamento.

10. Comparacao sobre Nomes, Escopo e Tempo de Vida

Versao Imperativa

Nomes declarados: caminho_arquivo, lista_alunos (globais); nome, media, i (locais).

Visibilidade: globais visiveis em todo o modulo; locais apenas dentro de suas funcoes.

Escopo: funcoes como calcular_media() tem escopo local; parametros nao vazam.

Tempo de vida: locais criadas na chamada e destruidas ao retornar; globais existem durante toda a execucao.

Versao Orientada a Objetos

Nomes declarados: classes Aluno e Turma; atributos self.__nome, self.__nota1; metodos calcular_media().

Visibilidade: atributos com __ (name mangling) visiveis apenas dentro da classe; metodos publicos acessiveis externamente.

Escopo: cada objeto tem seu proprio escopo de atributos; metodos acessam self.

Tempo de vida: objetos existem enquanto houver referencia; garbage collector os destroi quando sem referencias.

Versao Funcional

Nomes declarados: funcoes puras calcular_media(), verificar_aprovacao(), gerar_resultado(); variavel alunos no modulo.

Visibilidade: cada funcao ve apenas seus parametros e variaveis locais. Sem estado mutavel compartilhado.

Escopo: funcoes sao de escopo global; lambdas em filter() capturam variaveis do escopo envolvente (closure).

Tempo de vida: dados criados a cada chamada e descartados apos retorno; listas intermediarias vivem no escopo do modulo.

Versao Logica (Prolog)

Nomes declarados: predicados aluno/6, media/2, aprovado/1; variaveis logicas Nome, Media.

Visibilidade: fatos e regras sao globais; variaveis logicas existem apenas dentro de sua clausula.

Escopo: cada clausula e independente; a variavel Nome em aprovado(Nome) so existe naquela regra.

Tempo de vida: variaveis instanciadas durante unificacao e deixam de existir ao termino da consulta ou backtracking.

11. Comparacao sobre Tipos de Dados

Python (3 versoes)

Tipagem: dinamica e forte.

Tipos primitivos: int, float, str, bool.

Strings: tipo nativo str, imutavel.

Colecoes: listas, dicionarios, tuplas.

Objetos: sim (versao OO com classes Aluno e Turma).

Conversao de tipo: explicita com float(), int().

Erros de tipo: TypeError em tempo de execucao.

Prolog

Tipagem: sem tipos explicitos.

Tipos primitivos: atomos, numeros, variaveis logicas.

Strings: representadas como atomos (ana, bruno).

Colecoes: listas Prolog e termos compostos.

Objetos: nao existe o conceito.

Conversao de tipo: automatica via operador is.

Erros de tipo: falha na unificacao (nao gera excecao, apenas falha).

12. Comparacao entre os Paradigmas

Como a versao imperativa organiza a solucao?

Sequencialmente: le dados, percorre com laço while, calcula media, verifica condicao, imprime. O programador controla cada passo.

Como a versao orientada a objetos organiza a solucao?

Modela o dominio: Aluno encapsula dados e comportamento, Turma gerencia a colecao. A logica e distribuida em metodos.

Como a versao funcional organiza a solucao?

Como pipeline de transformacoes: ler_dados -> map(gerar_resultado) -> imprimir_relatorio. Dados fluem entre funcoes puras.

Como a versao declarativa organiza a solucao?

Declara fatos (dados dos alunos) e regras (media, aprovacao). O motor de inferencia de Prolog encontra as respostas.

Qual versao ficou mais proxima da forma humana?

Prolog. A regra aprovado(Nome) :- media(Nome, M), M >= 6.0 e quase linguagem natural.

Qual versao ficou mais proxima da maquina?

A imperativa. O laço while com indice manual e atribuicoes explicitas reflete como o processador executa instrucoes.

Qual versao parece mais adequada para esse problema?

A funcional. O problema e essencialmente uma transformacao de dados (entrada -> processamento -> saida), caso ideal do paradigma funcional.

13. Conclusao

Cada paradigma demonstrou forcas e limitacoes claras ao resolver o mesmo problema:

O paradigma imperativo e o mais intuitivo para iniciantes e oferece controle total sobre o fluxo, mas gera codigo mais verboso e propenso a erros de indice.

O paradigma orientado a objetos brilha em organizacao e manutenibilidade. Encapsulamento protege os dados e facilita extensoes futuras (ex.: adicionar novos tipos de aluno). E o mais adequado para sistemas grandes.

O paradigma funcional produziu o codigo mais conciso e testavel. Funcoes puras sem efeito colateral facilitam depuracao e paralelismo. E ideal para problemas de transformacao de dados.

O paradigma logico e o mais declarativo e conciso. O programador descreve o que quer, nao como fazer. E poderoso para regras de negocio e sistemas especialistas, mas limitado para I/O e interfaces.

Nao existe paradigma universalmente "melhor". A escolha depende do problema: sistemas corporativos favorecem OO; pipelines de dados favorecem funcional; regras de inferencia favorecem logico; scripts simples favorecem imperativo. O programador competente domina multiplos paradigmas e escolhe o mais adequado para cada situacao.